XPERT CHAMPS | AI ENGINEERING INTERNSHIP

AI Personalized Learning Coach

Project Report and AI-Assisted Development Log

Python · Streamlit · Gemini API · SQLite

Created by Madhu

October 2026

1. Project Overview

The **AI Personalized Learning Coach** is a web application that builds a study plan around a student's goal, current level and available time. A Large Language Model (Google Gemini) writes the plan, the quizzes and the coding exercises. Python then checks every answer itself, finds the topics that need more practice, recommends what to do next, and stores everything in SQLite so progress is still there after the app restarts.

It is more than a chatbot: it has a complete study workflow (setup, plan, quiz, scoring, feedback, weak-topic detection, progress dashboard) and a skill-based coding practice module.

Objectives

- Collect the learner's subject, goal, level, target duration and daily study time.
- Generate a structured day-by-day study plan and validate it before use.
- Generate topic quizzes, score them in Python and explain each answer.
- Flag weak topics and recommend revision, more practice or the next topic.
- Save plans, quiz attempts, code attempts and progress in SQLite.
- Show everything in a clear, professional Streamlit dashboard.

> **Key design rule:** the LLM writes content, but **Python does the checking**. Quiz scores come from the answer key, and code is judged by running test cases. The model never calculates a score or decides pass or fail.

2. Features and Requirement Coverage

ID	Requirement	How it is implemented
FR-01	Learner setup	Learner Setup form with validation (subject, goal, level, days, minutes per day)
FR-02	Generate learning plan	Gemini returns JSON; Python validates day count, topics, objectives, time
FR-03	View and adjust plan	Study Plan page; regenerate with new goal or schedule; switch between old plans
FR-04	Topic selection	Topic dropdown on Quiz and Code Practice pages; buttons on each plan day
FR-05	Quiz generation	Five multiple-choice questions per topic in structured JSON
FR-06	Answer submission	Radio buttons per question; score calculated by Python from the answer key
FR-07	Feedback	Per-question explanations; optional AI coaching tip on request
FR-08	Identify weak topics	Below 60% marks the topic as 'needs revision'
FR-09	Recommend next steps	Revise / practise again / move on; dashboard shows one recommended next step
FR-10	Save progress	SQLite tables for learners, plans, topics, quiz attempts, code attempts
FR-11	Progress dashboard	KPI cards, progress ring, roadmap, history chart, code practice history
FR-12	Error handling	Validation messages, retries, model fallback, friendly API and database errors

Extra features beyond the brief

- **Code Practice:** Python exercises whose difficulty follows the learner's skill; code is run against test cases.
- **Adaptive quizzes:** the next quiz on a topic becomes easier after a weak score and harder after a strong one.
- **Professional dark theme** with a branded sidebar, KPI cards and roadmap view.
- **Resilience:** streaming replies, automatic fallback to another model, and a `check_api.py` diagnostic tool.

3. Technology Stack

Technology	Purpose
Python 3.12	Application logic, validation, quiz scoring, code test runner
Streamlit	Web user interface and dashboard
Google Gemini API (google-genai SDK)	Generates plans, quizzes, exercises and coaching tips
SQLite (sqlite3)	Stores learner profile, plans, topics, quiz and code attempts
python-dotenv	Loads the API key from the .env file
pandas	Tables and charts on the dashboard
unittest	32 automated tests
Git and GitHub	Version control and submission

4. System Design

Application workflow

- The learner opens the app and fills in the setup form; inputs are validated.
- The app sends the profile to Gemini and asks for a structured study plan in JSON.
- Python validates the plan (exact number of days, objectives, time limit) and saves it to SQLite.
- The learner picks a topic and generates a quiz or a coding exercise.
- Python scores the quiz from the answer key, or runs the code against test cases.
- The attempt is saved, the topic status is updated and a recommendation is shown.
- The dashboard shows progress, weak topics, history and the next step.

Project files

File	Responsibility
app.py	Streamlit pages: Home, Learner Setup, Study Plan, Practice Quiz, Code Practice, Progress Dashboard
ai_service.py	Gemini calls, JSON parsing, retries, streaming, model fallback, re-prompting on invalid output
learning_plan.py	Input validation, plan prompt, plan validation
quiz_manager.py	Quiz prompt and validation, Python scoring, weak-topic logic, recommendations

File	Responsibility
code_manager.py	Exercise prompt and validation, sandboxed test runner, skill-based difficulty
database.py	SQLite schema and queries
ui_theme.py	Dark professional theme: CSS, KPI cards, progress ring, roadmap rows, footer
check_api.py	Tests which Gemini models work for the learner's key and how fast
tests/test_core.py	Automated tests (32)

5. Database Design (SQLite)

Table	Main columns
learners	id, subject, goal, level, duration_days, daily_minutes, created_at
plans	id, learner_id (FK), title, summary, is_active, created_at
topics	id, plan_id (FK), day, title, objectives_json, activities_json, estimated_minutes, status, attempts, last_score, best_score, updated_at
quiz_attempts	id, topic_id (FK), plan_id (FK), score, total, percentage, questions_json, answers_json, created_at
code_attempts	id, topic_id (FK), plan_id (FK), title, difficulty, passed, total, code, created_at

Every quiz or code attempt is saved as a **new row**, so the full history is kept. Regenerating a plan creates a new plan and keeps the old one. Topic status values are: pending, in_progress, completed and needs_revision.

6. LLM Integration and Prompt Design

What the application asks the model to do

Prompt	Asks the model for	Python then...
Study plan	Exactly N days with topic, 2-3 short objectives, estimated minutes (not above daily time) and activities, as JSON	Checks day count, fields and time limit; caps minutes; re-prompts with the error if invalid (up to 3 tries)
Quiz	Five multiple-choice questions, four options, a letter answer key, one-sentence explanations. Difficulty depends on the last score	Validates structure, shuffles options (removes position bias) and scores the answers itself
Coding exercise	One function, 4-6 test cases, starter code, hint and a reference solution, matched to the learner's skill	Runs the reference solution against the tests; rejects the exercise if it fails or the starter already solves it
Coaching tip	2-4 sentences on what to review (only when the learner presses the button)	Shows the text; never uses it for scoring

Rules built into every prompt

- Follow the learner's subject, level, duration and daily study time.
- Return JSON only, in a fixed structure that Python can validate.
- Give explanations suited to the learner's level.

- Never claim mastery from one small quiz (a strong first score gets a 'revisit later' note).

Quiz scoring and recommendations

Score	Topic status	Recommendation
Below 60%	needs_revision	Revise the topic, then retake a quiz
60% to 79%	in_progress	Review missed questions and try one more quiz
80% or more	completed	Move on or try harder practice; revisit later to confirm

Speed and reliability

- Streaming responses and minimal model 'thinking' for faster replies; shorter prompts so the model writes less.
- A 90-second timeout, one retry per model, then automatic fallback to another available flash model.
- Quiz results appear instantly: the AI coaching tip is fetched only when requested.

7. Code Practice Module

- **Skill-based difficulty:** starts at the learner's level (Beginner = easy, Intermediate = medium, Advanced = hard), goes up after a perfect run and down after a weak one. The learner can override it.
- **Verified exercises:** the model's own solution must pass its own tests before the learner sees the exercise.
- **Safe test runner:** learner code runs in a separate Python process with an 8-second time limit and a minimal environment (the API key is not passed to it). Return values are compared with the test cases in Python.
- **Feedback:** each test shows expected output versus actual output; the reference solution can be revealed afterwards.

> **Note:** code runs on the machine hosting the app. On a public deployment, strangers could run code on the server, so keep the link private or hide the Code Practice page.

8. Error Handling

Situation	Behaviour
Missing subject or goal	Clear messages ask the learner to complete the fields
Invalid or malformed model reply	Automatic re-prompt including the validation error; friendly error after 3 failures; no crash
API timeout, overload or model not found	Retry, then fall back to another model; message explains what to do and how to run check_api.py
Wrong or missing API key	Immediate clear message, no pointless retries
Database error	Friendly message instead of a crash
Quiz submitted with unanswered questions	Warning; nothing is saved until all are answered
Learner code crashes, loops forever or has a syntax error	Reported as a test result; time limit stops infinite loops

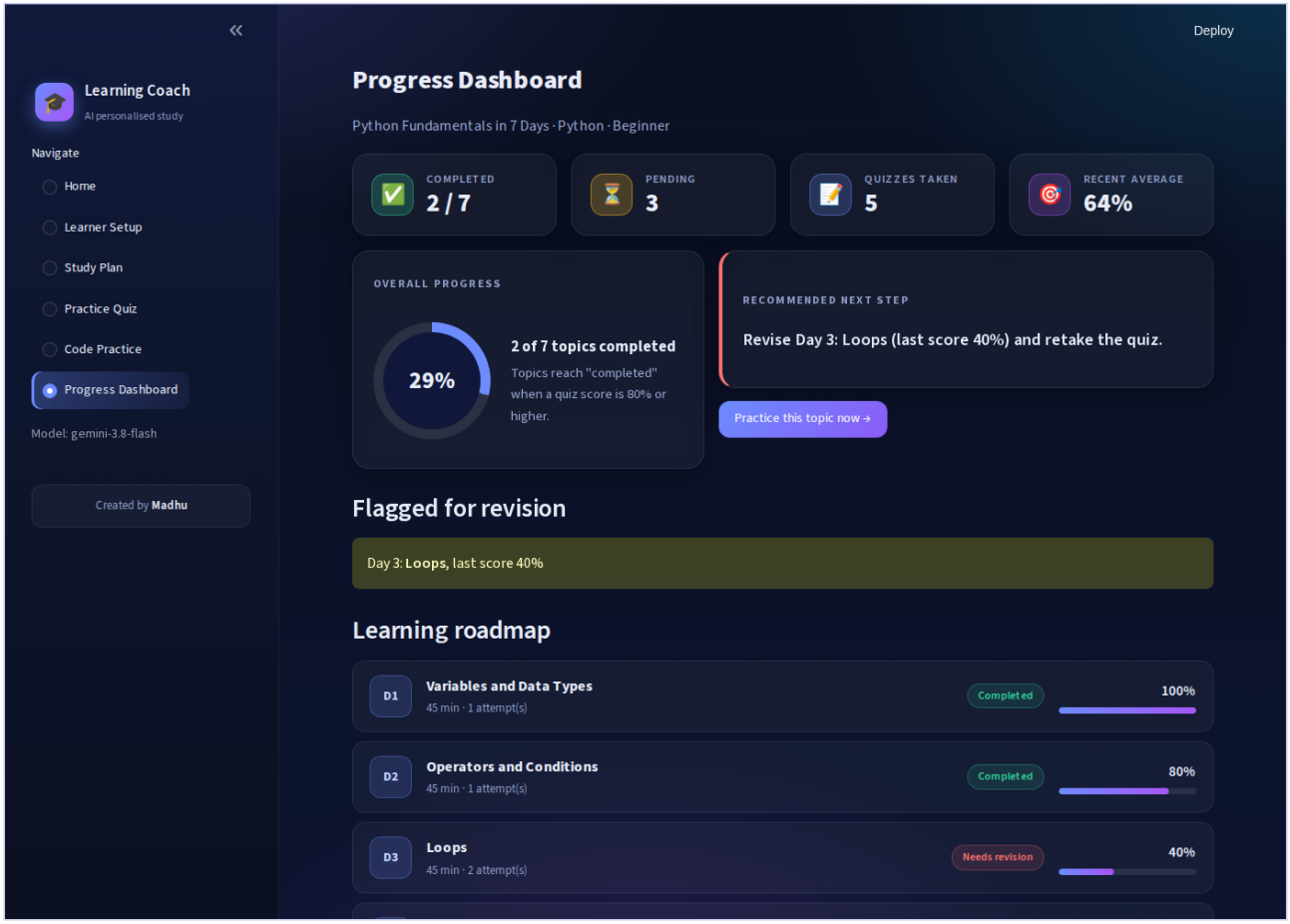
9. Testing

The project has **32 automated tests** (run with `python -m unittest discover -s tests -v`). They use mocked LLM calls, so no API key is needed. The full user interface was also tested end to end with a stubbed model.

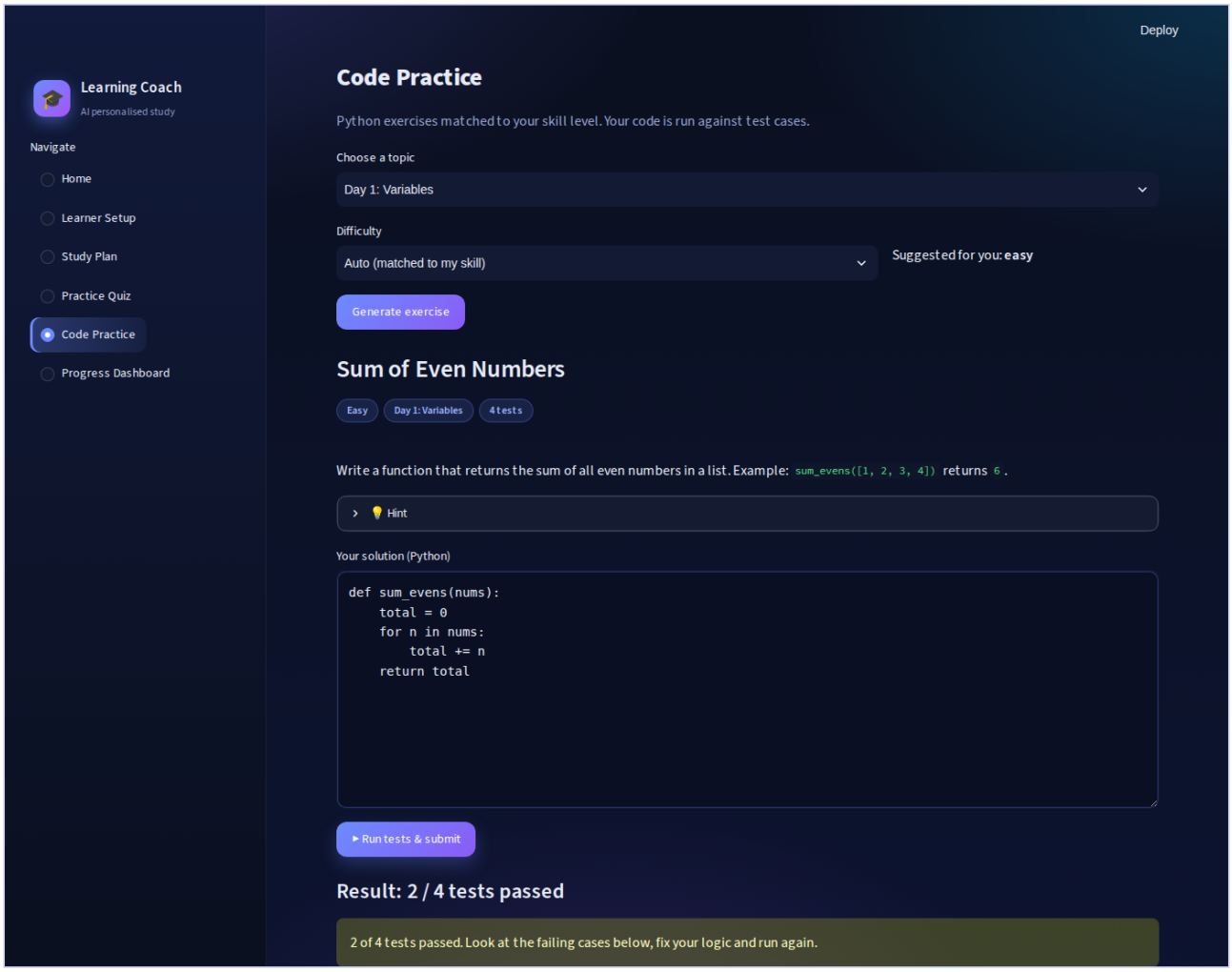
Test case from the brief	Expected behaviour	Covered by
Missing subject or goal	Ask for required fields	Profile validation tests
Valid learner setup	Plan matches level and time	Plan validation tests (time capped)
Invalid LLM response	Error or safe retry, no crash	Retry and invalid-reply tests
Quiz answer submission	Score from the answer key	Scoring tests (including shuffled options)
Low / high quiz score	Revision / move forward	Recommendation and status tests
Database persistence	Data remains after restart	Persistence test (re-open database)
API failure	Helpful message and retry	API failure and model fallback tests
Repeated quiz attempt	Each attempt saved separately	Repeated attempts test
Code practice (extra)	Right/wrong code, errors, infinite loops	Test runner and exercise validation tests

10. Screenshots

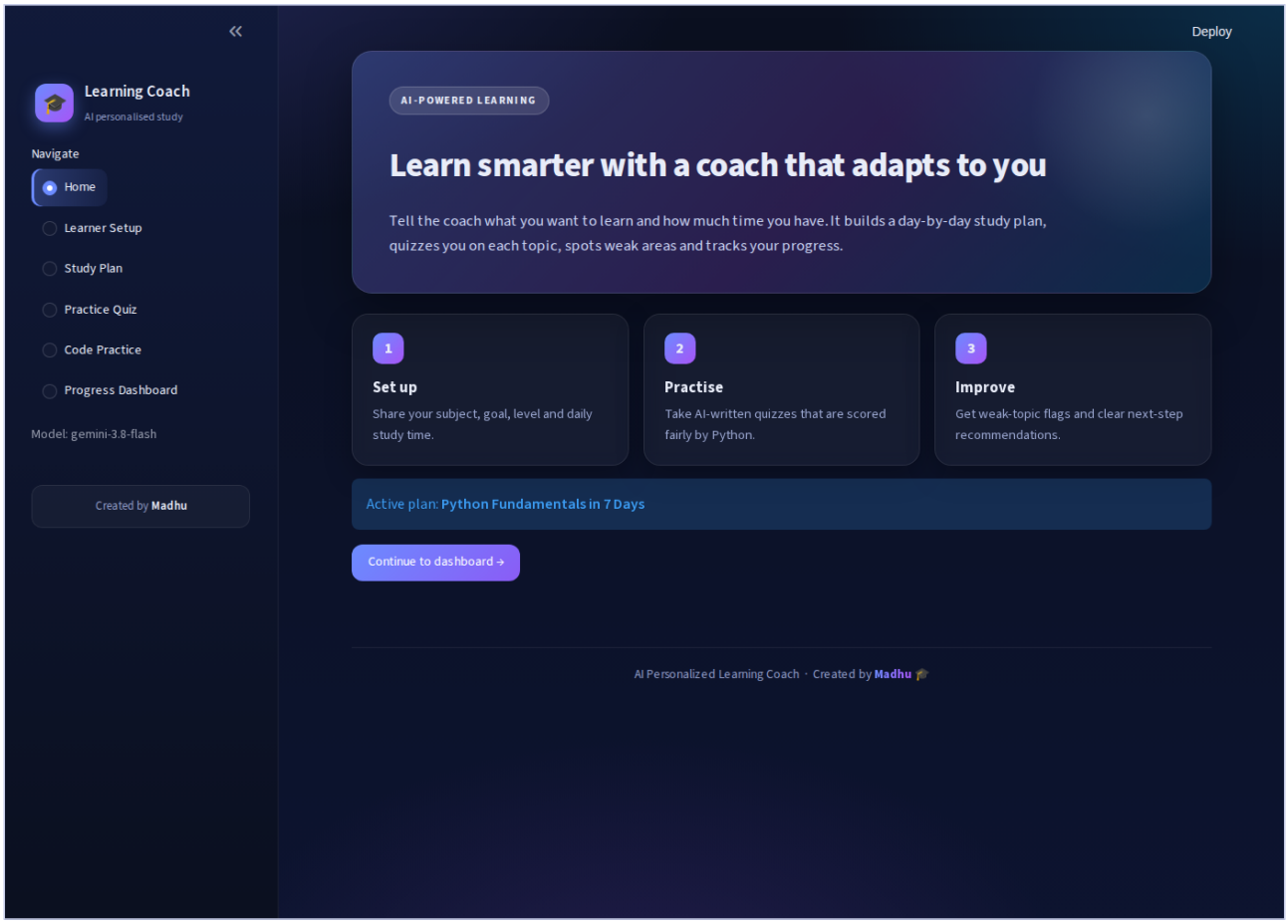
Dashboard and home page below use demo data created for the screenshots.



Progress Dashboard: KPI cards, progress ring, recommended next step and learning roadmap



Code Practice: skill-matched exercise, code editor and test results



Home page with branded sidebar and footer

11. Limitations and Future Improvements

Limitations

- AI-written quiz questions can occasionally be wrong; there is no human review of the answer key.
- Five questions is a small sample, so a score is only a rough signal of understanding.
- Single local user with no login; data lives in one SQLite file on the machine running the app.
- Free API tiers have rate limits, and a model can be slow or overloaded at busy times.
- Code Practice supports Python only and is not a hardened sandbox.

Future improvements

- User accounts and a hosted database so progress is private and permanent.
- A spaced-repetition schedule that brings weak topics back automatically.
- More languages for code practice, run in a stronger sandbox (for example a container).
- Exportable progress reports and charts per topic.

12. How to Run and Deploy

Run on Windows (PowerShell)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
copy .env.example .env
notepad .env   (add GEMINI_API_KEY)
python check_api.py   (optional: find the fastest model)
python -m streamlit run app.py
```

Deploy

- Push the project to GitHub (the .env file and database are git-ignored, so the key is not uploaded).
- On share.streamlit.io choose the repository, branch main and file app.py.
- Add GEMINI_API_KEY (and GEMINI_MODEL) in the app's Secrets, then click Deploy.
- On the free host the SQLite file is temporary and shared by all visitors; use a private link for demos.

13. How AI Was Used: Queries Used to Develop This Project

This project was developed step by step in a conversation with an AI assistant (Claude by Anthropic). The table lists the queries I used, in order, written exactly as I typed them, with what each one produced. The pasted error messages are shortened.

#	Stage	My query (as typed)	What the AI produced
1	Build the project	`[Uploaded the project brief PDF] this is my plan for my project according that write a program and create my project`	Complete project: app.py, ai_service.py, learning_plan.py, quiz_manager.py, database.py, tests, README, requirements, .env.example, .gitignore, sample data
2	Fix setup error	`PS C:\Users\admin\...\ai-personalized-learning-coach> streamlit run app.py streamlit : The term 'streamlit' is not recognized as the name of a cmdlet...`	Explained the missing install / PATH problem; gave virtual-environment steps and the python -m streamlit run command
3	Fix model error	`Could not get a response from the AI service... ClientError: 404 NOT_FOUND ... This model models/gemini-2.5-flash is no longer available to new users...`	Updated the default model name and explained that .env overrides the code default
4	Fix slow response	`Designing your personalised study plan... it taking so much time to responce`	Added a request timeout, fewer retries and a tip to test with a short plan first
5	Improve the design	`dash board is not good we want best backuground like profesional`	Dark gradient theme, KPI cards, progress ring, learning roadmap, branded sidebar (ui_theme.py, config.toml)
6	Speed and new feature	`we want fast answering and for practice codes codes for based on skills`	Minimal-thinking mode, shorter prompts and instant quiz results; new Code Practice page with skill-based difficulty and a tested code runner (code_manager.py)
7	Fix timeout error	`Could not get a response from the AI service... ServerError: 504 DEADLINE_EXCEEDED ... Deadline expired before operation could complete.`	Streaming replies, automatic fallback to another model, clearer errors and the check_api.py tool
8	Add credit	`add created my madhu down side`	'Created by Madhu' in the sidebar and as a footer on every page
9	Get commands	`give me commands`	Windows PowerShell commands for setup, run, tests and Git
10	Deployment	`how to deploye this`	Streamlit Community Cloud steps plus warnings about data, shared access and public code execution
11	Documentation	`create pdf according this project and what queris iam used to developthis project using ai`	This report

What worked well

- **Start with the brief.** Attaching the full project brief gave the AI the exact requirements to build against.

- **Paste the complete error message.** Queries 2, 3 and 7 were fixed quickly because the real error text was included.
- **One change at a time.** Each query asked for a single improvement (design, speed, a feature, a credit line), which kept the code working.
- **Test after every change.** Each version was run again and the automated tests were kept passing.

Responsible use

The code was generated with AI assistance, but I ran it, tested it and can explain it. For the final viva I should be able to explain the LLM integration, prompt design, quiz scoring, database structure, error handling and limitations in my own words (see the next section).

14. Viva Preparation: Likely Questions

Question	Short answer
Why is the score not calculated by the LLM?	LLMs can make arithmetic or judging mistakes. Python compares answers with the answer key, so scoring is always exact and repeatable.
How do you handle a bad AI reply?	The reply is parsed and validated. If invalid, the app re-prompts with the error message (up to 3 tries) and then shows a friendly error instead of crashing.
Why shuffle the quiz options in Python?	LLMs tend to put the correct answer in the same position; shuffling removes that bias while keeping the answer key aligned.
How does the app adapt to the learner?	Weak scores flag topics for revision and make the next quiz easier; strong scores move the learner on and make it harder. Code difficulty follows level and past results.
How do you make sure code exercises are correct?	The model's reference solution is run against its own tests before the learner sees the exercise; wrong ones are rejected.
What does your database store?	Learners, plans, topics with status and scores, every quiz attempt and every code attempt, so history survives a restart.
What are the limitations?	AI questions can be wrong, five questions is a small sample, single user, free API limits, and code running is not a hardened sandbox.